Complete DSA Cheatsheet

26 Problems - Trees, Graphs, Backtracking, DP

Core Concepts

Dynamic Programming

DP = Recursion + Memoization

- Optimal substructure
- Overlapping subproblems
- Store & reuse results

Top-Down: Problem→base (recursion+memo)
Bottom-Up: Base→problem (iteration+dp[])

Tree Traversal

DFS: Recursion/Stack, O(h) space
BFS: Queue, O(w) space

Preorder: Root, Left, Right
Postorder: Left, Right, Root
Level Order: BFS layer by layer

Backtracking

Template: Try choice → Recurse → Undo

- Explore all possibilities
- Prune invalid branches
- Build solution incrementally

1. Climbing Stairs (LeetCode 70)

Problem Statement

Given: n stairs, can climb 1 or 2 steps at a time
Find: Number of distinct ways to reach top

Example 1:
Input: n = 2
Output: 2
Explanation: 1+1, 2

Example 2:
Input: n = 3
Output: 3
Explanation: 1+1+1, 1+2, 2+1

Approach 1: Pure Recursion

Intuition

$f(n) = f(n-1) + f(n-2)$ (Fibonacci)
To reach step n: came from n-1 or n-2

```
int climbStairs(int n) {
    if (n == 0 || n == 1) return 1;
    return climbStairs(n-1) + climbStairs(n-2);
}
```

Complexity

Time: $O(2^n)$ **Space:** $O(n)$
Recalculates subproblems exponentially

Approach 2: Memoization (Top-Down)

```
int climbStairs(int n) {
    int[] memo = new int[n + 1];
    return helper(n, memo);
}

int helper(int n, int[] memo) {
    if (n == 0 || n == 1) return 1;
    if (memo[n] != 0) return memo[n];
    memo[n] = helper(n-1, memo) + helper(n-2, memo);
    return memo[n];
}
```

Complexity

Time: $O(n)$ **Space:** $O(n)$
Each subproblem solved once, memo array + stack

Approach 3: Bottom-Up (Iterative)

```
int climbStairs(int n) {
    if (n <= 1) return 1;
    int[] dp = new int[n + 1];
    dp[0] = 1; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

Approach 4: Space-Optimized

```
int climbStairs(int n) {
    if (n <= 1) return 1;
    int prev2 = 1, prev1 = 1;
    for (int i = 2; i <= n; i++) {
        int cur = prev1 + prev2;
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Complexity

Time: $O(n)$ **Space:** $O(1)$
Only need last 2 values (fixed look-back)

Common Mistakes

× Base case returns 0 instead of 1
× Creating memo inside recursion
× Confusing input (stairs) with output (ways)

2. House Robber (LeetCode 198)

Problem Statement

Given: Array representing amount of money in houses
Constraint: Can't rob adjacent houses
Find: Maximum amount of money can rob

Example 1:
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (1) + house 3 (3) = 4

Example 2:
Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (2) + house 3 (9) + house 5 (1) = 12

Approach 1: Recursion + Memoization

```java
int rob(int[] nums) {
    int[] memo = new int[nums.length];
    Arrays.fill(memo, -1);
    return robFrom(0, nums, memo);
}

int robFrom(int i, int[] nums, int[] memo) {
    if (i >= nums.length) return 0;
    if (memo[i] != -1) return memo[i];
    int rob = nums[i] + robFrom(i+2, nums, memo);
    int skip = robFrom(i+1, nums, memo);
    memo[i] = Math.max(rob, skip);
    return memo[i];
}
```

Approach 2: Bottom-Up

```java
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);
    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(nums[i] + dp[i-2], dp[i-1]);
    }
    return dp[nums.length - 1];
}
```

Approach 3: Space-Optimized

```java
int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);
    for (int i = 2; i < nums.length; i++) {
        int cur = Math.max(nums[i] + prev2, prev1);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Common Mistakes

× Thinking it's even/odd pattern
✓ It's optimal choice at each house

3. Coin Change (LeetCode 322)

Problem Statement

Given: Integer array coins, integer amount
Find: Minimum number of coins to make amount
Return: -1 if impossible

Example 1:
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1

Example 2:
Input: coins = [2], amount = 3
Output: -1
Explanation: Cannot make 3 with only 2s

Approach 1: Recursion + Memoization

```java
int coinChange(int[] coins, int amount) {
    int[] memo = new int[amount + 1];
    Arrays.fill(memo, -1);
    int res = helper(coins, amount, memo);
    return res == Integer.MAX_VALUE ? -1 : res;
}

int helper(int[] coins, int amt, int[] memo) {
    if (amt == 0) return 0;
    if (amt < 0) return Integer.MAX_VALUE;
    if (memo[amt] != -1) return memo[amt];
    int min = Integer.MAX_VALUE;
    for (int coin : coins) {
        int res = helper(coins, amt - coin, memo);
        if (res != Integer.MAX_VALUE) {
            min = Math.min(min, 1 + res);
        }
    }
    memo[amt] = min;
    return min;
}
```

Approach 2: Bottom-Up

```java
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], 1 + dp[i - coin]);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Common Mistakes

× Using -1 for impossible during recursion
✓ Use MAX_VALUE, convert to -1 at end
× Not understanding "1+" is coin just used
× Using Integer.MAX_VALUE causes overflow

4. 0/1 Knapsack Problem

Problem Statement

Given: n items with weights[], values[], capacity W
Constraint: Each item used 0 or 1 times
Find: Maximum value without exceeding capacity

Example:
Input: weights = [1,3,4,5], values = [1,4,5,7], W = 7
Output: 9
Explanation: Take items with weight 3 and 4 (values 4+5=9)

Approach 1: Recursion + Memoization

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] memo = new int[n][W + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dfs(W, val, wt, 0, memo);
}

int dfs(int W, int[] val, int[] wt, int i, int[][] memo) {
    if (i >= wt.length || W == 0) return 0;
    if (memo[i][W] != -1) return memo[i][W];
    int skip = dfs(W, val, wt, i + 1, memo);
    int take = 0;
    if (wt[i] <= W) {
        take = val[i] + dfs(W - wt[i], val, wt, i + 1, memo);
    }
    memo[i][W] = Math.max(skip, take);
    return memo[i][W];
}
```

Approach 2: 2D Bottom-Up

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            int skip = dp[i-1][w];
            int take = 0;
            if (wt[i-1] <= w) {
                take = val[i-1] + dp[i-1][w - wt[i-1]];
            }
            dp[i][w] = Math.max(skip, take);
        }
    }
    return dp[n][W];
}
```

Approach 3: 1D Space-Optimized

```java
int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[] dp = new int[W + 1];
    for (int i = 0; i < n; i++) {
        // BACKWARD iteration!
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
    return dp[W];
}
```

Common Mistakes

× Forward iteration (allows reuse)
✓ Backward iteration (0/1 property)
× dp[i] uses array[i] (wrong indexing)
✓ dp[i] uses array[i-1] (1-indexed DP)

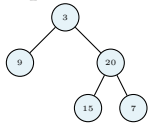
5. Binary Tree Level Order (LeetCode 102)

Problem Statement

Given: Root of binary tree
Find: Level order traversal (nodes grouped by level)

Example:
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Input Tree:



Approach: BFS with Queue

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();  // MUST capture!
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Complexity

Time: $O(n)$ **Space:** $O(w)$ (width)
Visit each node once. Queue holds max width.

Common Mistakes

× Not capturing size before loop
× Reusing list outside loop
✓ New list each iteration

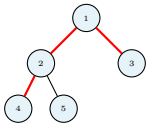
6. Diameter of Binary Tree (Leet-Code 543)

Problem Statement

Given: Root of binary tree
Find: Length of diameter (longest path between any 2 nodes)
Note: Path may or may not pass through root

Example:
Input: root = [1,2,3,4,5]
Output: 3
Explanation: Path 4→2→1→3 has length 3

Input Tree:



Red path = diameter

Approach 1: Global Variable

```
class Solution {
    int diameter = 0;

    int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }

    int height(TreeNode node) {
        if (node == null) return 0;
        int left = height(node.left);
        int right = height(node.right);
        diameter = Math.max(diameter, left + right);
        return 1 + Math.max(left, right);
    }
}
```

Approach 2: Array Parameter

```
int diameterOfBinaryTree(TreeNode root) {
    int[] diameter = new int[1];
    height(root, diameter);
    return diameter[0];
}

int height(TreeNode node, int[] diameter) {
    if (node == null) return 0;
    int left = height(node.left, diameter);
    int right = height(node.right, diameter);
    diameter[0] = Math.max(diameter[0], left + right);
    return 1 + Math.max(left, right);
}
```

Complexity

Time: $O(n)$ **Space:** $O(h)$
Visit each once. Stack = height.

Common Mistakes

× Trying to return both height and diameter
✓ Return height, track diameter separately

7. Validate Binary Search Tree (Leet-Code 98)

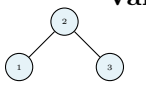
Problem Statement

Given: Root of binary tree
Find: Is it a valid BST?
Rule: Left < root < right for ALL descendants

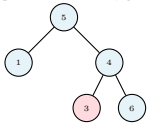
Example 1:
Input: root = [2,1,3]
Output: true

Example 2:
Input: root = [5,1,4,null,null,3,6]
Output: false (3 < 5)

Valid BST:



Invalid BST:



Approach: Recursive with Bounds

```
boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

boolean valid(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (!(min < node.val && node.val < max)) {
        return false;
    }
    return valid(node.left, min, node.val) &&
           valid(node.right, node.val, max);
}
```

Common Mistakes

× Using int min/max (node might be INT_MIN)
✓ Use Long.MIN/MAX for bounds
× Only comparing with immediate parent
✓ Pass bounds down recursively

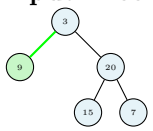
8. Minimum Depth of Binary Tree (LeetCode 111)

Problem Statement

Given: Root of binary tree
Find: Minimum depth (shortest path to leaf)

Example:
Input: root = [3,9,20,null,null,15,7]
Output: 2
Explanation: Shortest path: 3→9 (length 2)

Input Tree:



Green path = shortest (depth 2)

Approach: BFS (Optimal)

```java
int minDepth(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            // Leaf found!
            if (node.left == null && node.right == null) {
                return depth;
            }
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
    }
    return depth;
}
```

Why BFS?

BFS stops at first leaf (shortest path)
DFS would visit all nodes (unnecessary)

9. Maximum Depth of Binary Tree (LeetCode 104)

Problem Statement

Given: Root of binary tree
Find: Maximum depth (longest path to leaf)

Example:
Input: root = [3,9,20,null,null,15,7]
Output: 3
Explanation: Longest path: 3→20→15 or 3→20→7

Approach 1: Recursive (DFS)

```java
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    int left = maxDepth(root.left);
    int right = maxDepth(root.right);
    return 1 + Math.max(left, right);
}
```

Approach 2: Iterative (BFS)

```java
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
    }
    return depth;
}
```

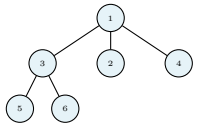
10. N-ary Tree Preorder Traversal (LeetCode 589)

Problem Statement

Given: Root of n-ary tree
Find: Preorder traversal (root before children)

Example:
Input: root = [1,null,3,2,4,null,5,6]
Output: [1,3,5,6,2,4]

Input N-ary Tree:



Output: [1,3,5,6,2,4]

Approach 1: Recursive

```java
List<Integer> preorder(Node root) {
    List<Integer> result = new ArrayList<>();
    preord(root, result);
    return result;
}

void preord(Node node, List<Integer> result) {
    if (node == null) return;
    result.add(node.val);
    for (Node child : node.children) {
        preord(child, result);
    }
}
```

Approach 2: Iterative (Stack)

```java
List<Integer> preorder(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.val);
        // Push RIGHT to LEFT (stack is LIFO)
        for (int i = node.children.size()-1; i >= 0; i--) {
            stack.push(node.children.get(i));
        }
    }
    return result;
}
```

Common Mistakes

× Pushing children left to right
✓ Push RIGHT to LEFT (stack reverses)

11. N-ary Tree Postorder Traversal (LeetCode 590)

Problem Statement

Given: Root of n-ary tree
Find: Postorder traversal (children before root)

Example:
Input: root = [1,null,3,2,4,null,5,6]
Output: [5,6,3,2,4,1]

Approach 1: Recursive

```java
List<Integer> postorder(Node root) {
    List<Integer> result = new ArrayList<>();
    postord(root, result);
```

```
    return result;
}

void postord(Node node, List<Integer> result) {
    if (node == null) return;
    for (Node child : node.children) {
        postord(child, result);
    }
    result.add(node.val);
}
```

Approach 2: Iterative (Reverse Trick)

```
List<Integer> postorder(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Stack<Node> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        Node node = stack.pop();
        result.add(node.val);
        // Push LEFT to RIGHT
        for (int i = 0; i < node.children.size(); i++) {
            stack.push(node.children.get(i));
        }
    }
    Collections.reverse(result);  // Reverse!
    return result;
}
```

Trick

Modified preorder + reverse = postorder

12. Maximum Depth of N-ary Tree (LeetCode 559)

Problem Statement

Given: Root of n-ary tree
Find: Maximum depth

Example:
Input: root = [1,null,3,2,4,null,5,6]
Output: 3
Explanation: Path 1→3→5 has depth 3

Approach 1: Recursive

```
int maxDepth(Node root) {
    if (root == null) return 0;
    int max = 0;
    for (Node child : root.children) {
        max = Math.max(max, maxDepth(child));
    }
    return max + 1;
}
```

Approach 2: Iterative (BFS)

```
int maxDepth(Node root) {
    if (root == null) return 0;
    Queue<Node> q = new LinkedList<>();
    q.offer(root);
    int depth = 0;

    while (!q.isEmpty()) {
        depth++;
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node node = q.poll();
            for (Node child : node.children) {
                q.offer(child);
            }
        }
    }
    return depth;
}
```

13. N-ary Tree Level Order (LeetCode 429)

Problem Statement

Given: Root of n-ary tree
Find: Level order traversal

Example:
Input: root = [1,null,3,2,4,null,5,6]
Output: [[1],[3,2,4],[5,6]]

Approach: BFS

```
List<List<Integer>> levelOrder(Node root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<Node> q = new LinkedList<>();
    q.offer(root);

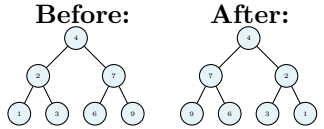
    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            Node node = q.poll();
            level.add(node.val);
            for (Node child : node.children) {
                q.offer(child);
            }
        }
        result.add(level);
    }
    return result;
}
```

14. Invert Binary Tree (LeetCode 226)

Problem Statement

Given: Root of binary tree
Find: Inverted tree (swap left and right)

Example:
Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]



Approach 1: Recursive

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);
    invertTree(root.right);
    return root;
}
```

Approach 2: Iterative (BFS)

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

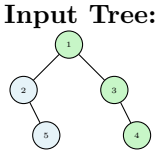
    while (!q.isEmpty()) {
        TreeNode node = q.poll();
        TreeNode temp = node.left;
        node.left = node.right;
        node.right = temp;
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    return root;
}
```

15. Binary Tree Right Side View (LeetCode 199)

Problem Statement

Given: Root of binary tree
Find: Values of rightmost node at each level

Example:
Input: root = [1,2,3,null,5,null,4]
Output: [1,3,4]



Green nodes = right side view

Approach: BFS

```java
List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        TreeNode rightSide = null;
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node != null) {
                rightSide = node;
                q.offer(node.left);
                q.offer(node.right);
            }
        }
        if (rightSide != null) res.add(rightSide.val);
    }
    return res;
}
```

16. Lowest Common Ancestor (LeetCode 236)

Problem Statement

Given: Binary tree, two nodes p and q
Find: Lowest common ancestor (LCA)

Example:
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: LCA of 5 and 1 is 3

Approach: Recursive

```java
TreeNode lowestCommonAncestor(TreeNode root,
    TreeNode p, TreeNode q) {
    if (root == null) return null;
    if (root == p || root == q) return root;

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

3 Cases

Both in left subtree → return left
Both in right subtree → return right
Split across subtrees → return root

17. Reverse Linked List (LeetCode 206)

Problem Statement

Given: Head of singly linked list
Find: Reversed list

Example:
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Approach 1: Iterative

```java
ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Complexity

Time: $O(n)$ **Space:** $O(1)$

Approach 2: Recursive

```java
ListNode reverseList(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseList(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

Complexity

Time: $O(n)$ **Space:** $O(n)$ (stack)

Common Mistakes

× newHead changes per frame
✓ newHead is SAME, head is different

18. Number of Islands (LeetCode 200)

Problem Statement

Given: m×n 2D grid, '1' = land, '0' = water
Find: Number of islands
Note: Island = connected horizontally/vertically

Example:
Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

Approach: DFS

```java
int numIslands(char[][] grid) {
    if (grid == null) return 0;
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
        }
    }
    return count;
}

void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 ||
        j >= grid[0].length || grid[i][j] == '0') {
        return;
    }
    grid[i][j] = '0';  // Mark visited
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

Complexity

Time: $O(m \times n)$ **Space:** $O(m \times n)$
Visit each cell. DFS depth = grid size.

Common Mistakes

× Not marking visited
✓ Mark as '0' to avoid revisiting

19. Max Area of Island (LeetCode 695)

Problem Statement

Given: m×n binary grid (1 = land, 0 = water)
Find: Maximum area of island

Example:
Input: grid = [[0,0,1,0,0],[0,1,1,0,0],[0,1,0,0,0]]
Output: 4
Explanation: Island of 4 connected 1s

Approach: DFS

```
int maxAreaOfIsland(int[][] grid) {
    int maxArea = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == 1) {
                int area = dfs(grid, i, j);
                maxArea = Math.max(maxArea, area);
            }
        }
    }
    return maxArea;
}

int dfs(int[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 ||
        j >= grid[0].length || grid[i][j] == 0) {
        return 0;
    }
    grid[i][j] = 0;
    return 1 + dfs(grid, i + 1, j)
             + dfs(grid, i - 1, j)
             + dfs(grid, i, j + 1)
             + dfs(grid, i, j - 1);
}
```

Key Difference

Return 1 + sum of neighbors (counts area)

20. Letter Combinations (LeetCode 17)

Problem Statement

Given: String of digits 2-9
Find: All possible letter combinations

Example:
Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
Explanation: 2→abc, 3→def

Approach: Backtracking

```
String[] map = {"", "", "abc", "def", "ghi",
    "jkl", "mno", "pqrs", "tuv", "wxyz"};

List<String> letterCombinations(String digits) {
    List<String> res = new ArrayList<>();
    if (digits.isEmpty()) return res;
    backtrack(0, "", digits, res);
    return res;
}

void backtrack(int i, String cur, String digits,
    List<String> res) {
    if (cur.length() == digits.length()) {
        res.add(cur);
        return;
    }
    String chars = map[digits.charAt(i) - '0'];
    for (char c : chars.toCharArray()) {
        backtrack(i + 1, cur + c, digits, res);
    }
}
```

Complexity

Time: $O(4^n)$ **Space:** $O(n)$
Each digit 4 letters. Depth = n.

Common Mistakes

× Explicit undo (strings immutable!)
✓ No undo needed, strings don't change

21. Permutations (LeetCode 46)

Problem Statement

Given: Array of distinct integers
Find: All possible permutations

Example:
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Approach: Backtracking

```
List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new ArrayList<>(),
        new boolean[nums.length], result);
    return result;
}

void backtrack(int[] nums, List<Integer> cur,
    boolean[] used, List<List<Integer>> result) {
    if (cur.size() == nums.length) {
        result.add(new ArrayList<>(cur));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        cur.add(nums[i]);
        used[i] = true;
        backtrack(nums, cur, used, result);
        cur.remove(cur.size() - 1);
        used[i] = false;
    }
}
```

Complexity

Time: $O(n!)$ **Space:** $O(n)$
n! permutations. Each O(n) to build.

Common Mistakes

× Not copying list when saving
× Forgetting to undo both add AND used
✓ Must undo both operations

22. Subsets (LeetCode 78)

Problem Statement

Given: Array of distinct integers
Find: All possible subsets (power set)

Example:
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Approach: Backtracking

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

void backtrack(int[] nums, int start,
    List<Integer> cur, List<List<Integer>> result) {
    result.add(new ArrayList<>(cur));  // Save at EVERY node!
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);
        backtrack(nums, i + 1, cur, result);
        cur.remove(cur.size() - 1);
    }
}
```

Complexity

Time: $O(2^n)$ **Space:** $O(n)$
2 subsets. Each O(n) to copy.

Common Mistakes

× Only saving at leaves
✓ Save at EVERY node (including empty)

23. Valid Palindrome (LeetCode 125)

Problem Statement

Given: String s
Find: Is it a palindrome? (alphanumeric only, ignore case)

Example 1:
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is palindrome

Example 2:
Input: s = "race a car"
Output: false

Approach: Two Pointers

```java
boolean isPalindrome(String s) {
    if (s == null || s.length() == 1) return true;
    int left = 0, right = s.length() - 1;

    while (left < right) {
        char l = s.charAt(left);
        char r = s.charAt(right);

        if (Character.isLetterOrDigit(l) &&
            Character.isLetterOrDigit(r)) {
            if (Character.toLowerCase(l) !=
                Character.toLowerCase(r)) {
                return false;
            }
            left++; right--;
        } else if (!Character.isLetterOrDigit(l)) {
            left++;
        } else {
            right--;
        }
    }
    return true;
}
```

Complexity

Time: $O(n)$ **Space:** $O(1)$

24. Longest Palindromic Substring (LeetCode 5)

Problem Statement

Given: String s
Find: Longest palindromic substring

Example 1:
Input: s = "babad"
Output: "bab" (or "aba")

Example 2:
Input: s = "cbbd"
Output: "bb"

Approach: Expand Around Center

```java
String longestPalindrome(String s) {
    int resLen = 0, resIdx = 0;
    for (int i = 0; i < s.length(); i++) {
        // Odd length
        int l = i, r = i;
        while (l >= 0 && r < s.length() &&
               s.charAt(l) == s.charAt(r)) {
            if (r - l + 1 > resLen) {
                resIdx = l; resLen = r - l + 1;
            }
            l--; r++;
        }
        // Even length
        l = i; r = i + 1;
        while (l >= 0 && r < s.length() &&
               s.charAt(l) == s.charAt(r)) {
            if (r - l + 1 > resLen) {
                resIdx = l; resLen = r - l + 1;
            }
            l--; r++;
        }
    }
    return s.substring(resIdx, resIdx + resLen);
}
```

Complexity

Time: $O(n^2)$ **Space:** $O(1)$
n centers, each expands n times.

25. Meeting Rooms (LeetCode 252)

Problem Statement

Given: Array of meeting time intervals
Find: Can person attend all meetings?

Example 1:
Input: intervals = [[0,30],[5,10],[15,20]]
Output: false
Explanation: [0,30] overlaps with others

Example 2:
Input: intervals = [[7,10],[2,4]]
Output: true

Approach: Sort + Check Consecutive

```java
boolean canAttendMeetings(List<Interval> intervals) {
    intervals.sort((a,b) -> a.start - b.start);
    for (int i = 1; i < intervals.size(); i++) {
        if (intervals.get(i).start < intervals.get(i-1).end) {
            return false;
        }
    }
    return true;
}
```

Complexity

Time: $O(n \log n)$ **Space:** $O(1)$
Sort dominates. In-place check.

26. Min Cost Climbing Stairs (LeetCode 746)

Problem Statement

Given: Integer array cost (cost[i] = cost of step i)
Find: Min cost to reach top (can start at 0 or 1)

Example:
Input: cost = [10,15,20]
Output: 15
Explanation: Start at index 1, pay 15, reach top

Approach 1: DP Array

```java
int minCostClimbingStairs(int[] cost) {
    int[] dp = new int[cost.length + 1];
    for (int i = 2; i <= cost.length; i++) {
        dp[i] = Math.min(cost[i-1] + dp[i-1],
                         cost[i-2] + dp[i-2]);
    }
    return dp[cost.length];
}
```

Approach 2: Space-Optimized

```java
int minCostClimbingStairs(int[] cost) {
    int prev2 = 0, prev1 = 0;
    for (int i = 2; i <= cost.length; i++) {
        int cur = Math.min(prev1 + cost[i-1],
                           prev2 + cost[i-2]);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Templates

Memoization

```
ReturnType solve(params) {
    Memo[] memo = initialize;
    return helper(params, memo);
}

ReturnType helper(params, Memo memo) {
    if (base_case) return base_value;
    if (memo.has(key)) return memo.get(key);
    result = compute using helper(smaller, memo);
    memo.store(key, result);
    return result;
}
```

Bottom-Up DP

```
ReturnType solve(params) {
    DP[] dp = initialize;
    dp[base] = base_value;
    for (int i = start; i <= end; i++) {
        dp[i] = compute using dp[smaller];
    }
    return dp[target];
}
```

BFS Level Order

```
Queue<Node> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();  // Capture!
    for (int i = 0; i < size; i++) {
        Node node = q.poll();
        for (Node child : node.children) {
            q.offer(child);
        }
    }
}
```

Backtracking

```
void backtrack(state, choices) {
    if (complete) { save(state); return; }
    for (choice : choices) {
        make_choice();
        backtrack(new_state);
        undo_choice();
    }
}
```

Common Mistakes

Critical Errors

× **Queue size in loop:** `for (i < q.size())`
✓ Capture first: `int size = q.size()`

× **Reuse list:** Create outside loop
✓ New list each iteration

× **Not copying:** `result.add(current)`
✓ `result.add(new ArrayList<>(current))`

× **Missing undo:** Add, recurse only
✓ Add, recurse, remove

× **Memo inside recursion:** Creates new each time
✓ Create once, pass to all

× **Forward 0/1:** Allows reuse
✓ Backward for 0/1 Knapsack

× **Base returns 0:** For counting problems
✓ Returns 1 (ways, not steps)

× **int bounds:** For BST validation
✓ Use Long.MIN/MAX

× **Return -1:** During recursion
✓ Use MAX_VALUE, convert at end

× **Index mapping:** dp[i] uses array[i]
✓ dp[i] uses array[i-1] (1-indexed DP)

Complexity Reference

Problem	Time	Space
Climb Stairs	$O(n)$	$O(1)$
House Robber	$O(n)$	$O(1)$
Coin Change	$O(amt \times c)$	$O(amt)$
Knapsack	$O(n \times W)$	$O(W)$
Level Order	$O(n)$	$O(w)$
Diameter	$O(n)$	$O(h)$
Validate BST	$O(n)$	$O(h)$
Min Depth	$O(n)$	$O(w)$
Max Depth	$O(n)$	$O(h)$
N-ary Pre/Post	$O(n)$	$O(h)$
N-ary Max	$O(n)$	$O(h)$
N-ary Level	$O(n)$	$O(w)$
Invert	$O(n)$	$O(h)$
Right View	$O(n)$	$O(w)$
LCA	$O(n)$	$O(h)$
Reverse List	$O(n)$	$O(1)/O(n)$
Islands	$O(mn)$	$O(mn)$
Max Area	$O(mn)$	$O(mn)$
Letter Comb	$O(4^n)$	$O(n)$
Permutations	$O(n!)$	$O(n)$
Subsets	$O(2^n)$	$O(n)$
Palindrome	$O(n)$	$O(1)$
Longest Pal	$O(n^2)$	$O(1)$
Meetings	$O(n \log n)$	$O(1)$
Min Cost	$O(n)$	$O(1)$

Quick Tips

Pattern Recognition

Counting (SUM): Climbing Stairs
Optimization (MAX/MIN): Robber, Coins, Knapsack
All combinations: Letter Comb, Perms, Subsets
Shortest path: BFS (Min Depth)
All paths: DFS (Diameter, Islands)
Validation: BST with bounds
Track + Return different: Diameter (global)
Unbounded: Coin Change (forward)
Bounded: Knapsack (backward)

Interview Strategy

1. Identify problem type
2. Start with brute force
3. Optimize with memo/DP
4. Discuss complexity
5. Consider space optimization
6. Test with examples
7. Handle edge cases